

LABORATORIO 1

Ejercicio 2: Departamentos ordenados por extensión

Código Java y sección del servidor

Tema	Departamentos ordenados ascendentemente por extensión
Operaciones	Alta, baja, modificación de precio, búsqueda y listado
Archivos	N_Departamento.java, Departamento.java y Servidor.java

Enunciado

Una inmobiliaria necesita almacenar los departamentos rentados ordenados ascendentemente por la extensión del departamento. Para cada departamento se registra: ubicación, extensión, precio, número de apartamento y nombre de la persona que lo rentó.

El programa permite dar de alta, dar de baja, modificar el precio por número, listar un departamento y listar todos los registros.

Estructura de archivos

- N_Departamento.java: guarda los datos de un departamento.
- Departamento.java: contiene el arreglo ordenado y sus operaciones.
- Servidor.java: conecta la interfaz HTML con las operaciones Java.

1. Clase N_Departamento.java

```
public class N_Departamento {
    private String ubicacion;
    private double extension;
    private double precio;
    private int numero;
    private String inquilino;

    public N_Departamento(String ubicacion, double extension, double precio, int numero, String
inquilino) {
        this.ubicacion = ubicacion;
        this.extension = extension;
        this.precio = precio;
        this.numero = numero;
        this.inquilino = inquilino;
    }

    public String getUbicacion() {
        return ubicacion;
    }

    public double getExtension() {
        return extension;
    }
}
```

```
public double getPrecio() {
    return precio;
}

public int getNumero() {
    return numero;
}

public String getInquilino() {
    return inquilino;
}

public void setPrecio(double precio) {
    this.precio = precio;
}

@Override
public String toString() {
    return ubicacion + " | " + extension + " m2 | C$ " + precio + " | Apto. " + numero + " | " +
    inquilino;
}
}
```

2. Clase Departamento.java

El arreglo se ordena ascendentemente por la extensión. El número del apartamento se utiliza para buscar, eliminar o modificar un departamento.

```
import java.util.Arrays;

public class Departamento {
    private N_Departamento[] departamentos;
    private int tam;

    public Departamento() {
        departamentos = new N_Departamento[100];
        tam = 0;
    }

    private int buscarNumero(int numero) {
        for (int i = 0; i < tam; i++) {
            if (departamentos[i].getNumero() == numero) {
                return i;
            }
        }

        return -1;
    }

    private int posicionInsertar(double extension) {
        int i = 0;

        while (i < tam && departamentos[i].getExtension() <= extension) {
            i++;
        }

        return i;
    }

    public String alta(String ubicacion, double extension, double precio, int numero, String
    inquilino) {
        if (tam == departamentos.length) {
            return "No hay espacio para más departamentos";
        }

        if (buscarNumero(numero) != -1) {
            return "El número de apartamento ya existe";
        }

        int pos = posicionInsertar(extension);
```

```
        for (int i = tam; i > pos; i--) {
            departamentos[i] = departamentos[i - 1];
        }

        departamentos[pos] = new N_Departamento(ubicacion, extension, precio, numero, inquilino);
        tam++;
        return "Departamento registrado correctamente";
    }

    public String baja(int numero) {
        int pos = buscarNumero(numero);

        if (pos == -1) {
            return "El departamento no existe";
        }

        for (int i = pos; i < tam - 1; i++) {
            departamentos[i] = departamentos[i + 1];
        }

        departamentos[tam - 1] = null;
        tam--;
        return "Departamento liberado correctamente";
    }

    public String modificarPrecio(int numero, double precio) {
        int pos = buscarNumero(numero);

        if (pos == -1) {
            return "El departamento no existe";
        }

        departamentos[pos].setPrecio(precio);
        return "Precio modificado correctamente";
    }

    public String listarUno(int numero) {
        int pos = buscarNumero(numero);

        if (pos == -1) {
            return "El departamento no existe";
        }

        return departamentos[pos].toString();
    }

    public String listarTodos() {
        if (tam == 0) {
            return "No hay departamentos registrados";
        }

        StringBuilder respuesta = new StringBuilder();

        for (int i = 0; i < tam; i++) {
            respuesta.append(departamentos[i].toString()).append("\n");
        }

        return respuesta.toString();
    }

    public N_Departamento[] getDepartamentos() {
        return Arrays.copyOf(departamentos, tam);
    }
}
```

3. Sección para Servidor.java

Agrega esta parte a tu servidor actual. Mantiene el mismo puerto 8080 y el mismo método de respuesta que usaste en el Ejercicio 1. Solo debes añadir la variable, las rutas y estos manejadores.

```
private static Departamento departamentos = new Departamento();
```

```
servidor.createContext("/ordenado2/alta", new AltaDepartamentoHandler());
servidor.createContext("/ordenado2/baja", new BajaDepartamentoHandler());
servidor.createContext("/ordenado2/modificar", new ModificarDepartamentoHandler());
servidor.createContext("/ordenado2/listarUno", new ListarUnoDepartamentoHandler());
servidor.createContext("/ordenado2/listarTodos", new ListarTodosDepartamentoHandler());

static class AltaDepartamentoHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }

        Map<String, String> datos = leerDatos(intercambio);
        String respuesta = departamentos.alta(
            datos.get("ubicacion"),
            Double.parseDouble(datos.get("extension")),
            Double.parseDouble(datos.get("precio")),
            Integer.parseInt(datos.get("numero")),
            datos.get("inquilino")
        );
        responder(intercambio, respuesta);
    }
}

static class BajaDepartamentoHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }

        Map<String, String> datos = leerDatos(intercambio);
        responder(intercambio, departamentos.baja(Integer.parseInt(datos.get("numero"))));
    }
}

static class ModificarDepartamentoHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }

        Map<String, String> datos = leerDatos(intercambio);
        String respuesta = departamentos.modificarPrecio(
            Integer.parseInt(datos.get("numero")),
            Double.parseDouble(datos.get("precio"))
        );
        responder(intercambio, respuesta);
    }
}

static class ListarUnoDepartamentoHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }

        Map<String, String> datos = leerDatos(intercambio);
        responder(intercambio, departamentos.listarUno(Integer.parseInt(datos.get("numero"))));
    }
}

static class ListarTodosDepartamentoHandler implements HttpHandler {
    @Override
    public void handle(HttpExchange intercambio) throws IOException {
        if (intercambio.getRequestMethod().equalsIgnoreCase("OPTIONS")) {
            responder(intercambio, "");
            return;
        }
    }
}
```

```
    }  
    responder(intercambio, departamentos.listarTodos());  
  }  
}
```

Rutas que utilizará la página HTML

El JavaScript del ejercicio debe enviar los datos por POST a estas rutas:

Operación	Ruta del servidor
Dar de alta	/ordenado2/alta
Dar de baja	/ordenado2/baja
Modificar precio	/ordenado2/modificar
Listar un departamento	/ordenado2/listarUno
Listar todos	/ordenado2/listarTodos

Importante para integrarlo

- Los campos de HTML deben llamarse: ubicacion, extension, precio, numero e inquilino.
- El arreglo no usa base de datos: los registros permanecen mientras el servidor Java está ejecutándose.
- Los departamentos se insertan desplazando elementos para conservar el orden ascendente por extensión.
- La búsqueda, baja y modificación se realizan por número de apartamento.



Ejercicio 2

Departamentos ordenados por extensión

Abrir ejercicio

Documento

Arreglos Ordenados: Ejercicio 2

Menú de Opciones

🌙 Modo Oscuro

Configurar arreglo

Tamaño del arreglo:

Ejemplo: 5

Iniciar ejercicio

Salir

Arreglos Ordenados: Ejercicio 2

Menú de Opciones

🌙 Modo Oscuro

1. Dar de alta

2. Dar de baja

3. Modificar precio

4. Listar un departamento

5. Listar todos

6. Salir



Arreglos Ordenados: Ejercicio 2

Menú de Opciones

 Modo Oscuro

1. Dar de alta

2. Dar de baja

3. Modificar precio

4. Listar un departamento

5. Listar todos

6. Salir

Dar de alta

Ubicación:

Ingrese la ubicación

Extensión en m²:

Ejemplo: 45

Precio:

Ejemplo: 450

Número de apartamento:

Ejemplo: 3

Nombre de la persona que renta:

Ingrese el nombre completo

Guardar

Arreglos Ordenados: Ejercicio 2

Menú de Opciones

 Modo Oscuro

1. Dar de alta

2. Dar de baja

3. Modificar precio

4. Listar un departamento

5. Listar todos

6. Salir

Dar de baja

Número de apartamento:

Arreglos Ordenados: Ejercicio 2

Menú de Opciones

 Modo Oscuro

1. Dar de alta

2. Dar de baja

3. Modificar precio

4. Listar un departamento

5. Listar todos

6. Salir

Modificar precio

Número de apartamento:

Nuevo precio:



Arreglos Ordenados: Ejercicio 2

Menú de Opciones

[🌙 Modo Oscuro](#)[1. Dar de alta](#)[2. Dar de baja](#)[3. Modificar precio](#)[4. Listar un departamento](#)[5. Listar todos](#)[6. Salir](#)

Listar un departamento

Número de apartamento:

[Buscar](#)

Lista de Departamentos Registrados

[← Regresar](#)[🌙 Modo Oscuro](#)

No hay departamentos registrados

Lista de Departamentos Registrados

[← Regresar](#)[🌙 Modo Oscuro](#)

#	Ubicación	Extensión	Precio	No. Apartamento	Persona que renta
1	Ana	45 m ²	450.0	3	Jose Santos Zelaya